CIÊNCIA DA COMPUTAÇÃO

AULA 06

Estrutura de Desvio, Operadores Lógicos e Estrutura de Condição

Linguagem C · Algoritmos · Programação Estruturada

Profª. Drª. Tatiele B. Moro

O QUE VEREMOS HOJE

01

Estrutura Sequencial vs Desvio

Limitações da sequência pura e necessidade de desvios

02

if / else / else if

Sintaxe e uso das estruturas de desvio em C

03

Operadores Relacionais

> < >= <= == != e suas aplicações práticas

04

Operadores Lógicos

&& (AND), || (OR), ! (NOT) com tabelas-verdade

05

Encadeamento de Condições

if aninhado e seleção encadeada

01

ESTRUTURA DE DESVIO

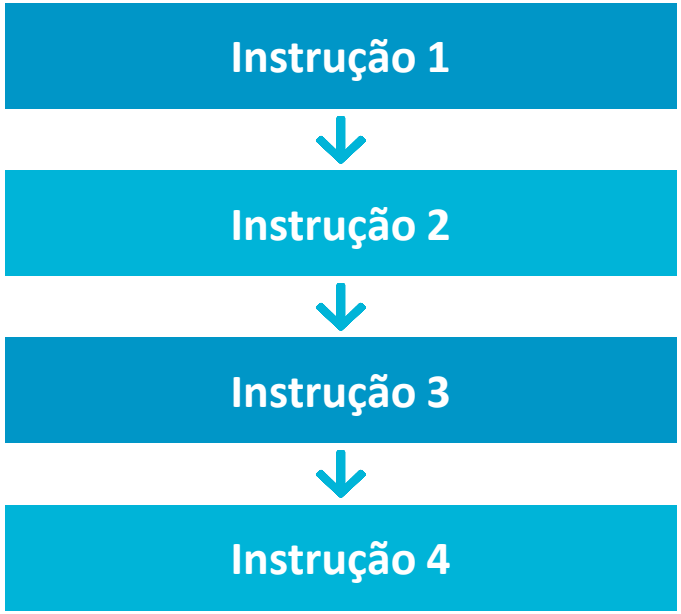
Controle de fluxo em programação C

Estrutura Sequencial

Como funciona?

Na estrutura sequencial, as instruções são executadas de **cima para baixo**, na ordem em que foram escritas, **uma após a outra**, sem nenhum desvio.

Limitação: nenhuma instrução é executada mais de uma vez. Isso restringe muito os problemas que podemos resolver.



Fluxo linear — sem escolhas

Estrutura de Desvio

O que é?

Permite desviar a execução de instruções conforme o valor de um dado, testando uma condição:

Condição VERDADEIRO	Condição FALSO
Condição?	
Um conjunto específico de instruções é executado	Outro conjunto é executado (ou o programa segue normalmente)

⚠ Em C, não existe tipo booleano: resultado ZERO = FALSO · resultado DIFERENTE DE ZERO = VERDADEIRO

Sintaxe: if e else em C

Desvio Simples (sem else)

```c
if (condição) {
    // instruções

}
```

Desvio com Alternativa (com else)

```c
if (condição) {
    // instruções se VERDADEIRO
}
else {
    // instruções se FALSO
}
```

Fluxo de decisão:

O que é a CONDIÇÃO?

É o resultado de uma operação com operadores relacionais e lógicos!

VERDADEIRO

Número diferente de zero

FALSO

Resultado igual a zero

Em C, não existe o tipo booleano!

0 → FALSO | qualquer outro número → VERDADEIRO

Exemplo: if (1) → sempre executa | if (0) → nunca executa

02

OPERADORES RELACIONAIS

Comparações que retornam verdadeiro ou falso

Operadores Relacionais

Geram resultados lógicos (verdadeiro/falso) pela comparação entre valores e/ou expressões

>

Maior que

10 > 2 → verdadeiro

<

Menor que

3 < 1 → falso

>=

Maior ou igual a

5 >= 5 → verdadeiro

<=

Menor ou igual a

4 <= 3 → falso

==

Igual a

x == y → compara!

!=

Diferente de

7 != 3 → verdadeiro

Todos os operadores relacionais possuem a MESMA prioridade entre si

Exemplos: Operadores > e >=

Operador > (estritamente maior)

```c
#include <stdio.h>
#include <conio.h>
main() {
  int idade;
  printf("Digite sua idade: ");
  scanf("%d", &idade);
  if (idade > 17) {
    printf("Maior de idade");
  }
}
```

17 → NÃO imprime · 18 → Imprime

Operador >= (maior ou igual)

```c
#include <stdio.h>
#include <conio.h>
main() {
  int idade;
  printf("Digite sua idade: ");
  scanf("%d", &idade);
  if (idade >= 18) {
    printf("Maior de idade");
  }
}
```

17 → NÃO imprime · 18 → Imprime

Resultado idêntico neste caso! Porém a lógica é diferente.

Exemplos: Operadores == e !=

== Igualdade com else

```
main() {
  int num1, num2;
  printf("Digite dois numeros: ");
  scanf("%d %d", &num1, &num2);
  if (num1 == num2) {
    printf("Os numeros sao iguais");
  }
  else {
    printf("Os numeros sao diferentes");
  }
}
```

!= Diferença

```
main() {
  int num1, num2;
  printf("Digite dois numeros: ");
  scanf("%d %d", &num1, &num2);
  if (num1 != num2) {
    printf("Os numeros sao
diferentes");
  }
}
```

Atenção: nunca use = no lugar de == para comparar!

Exemplo: Positivo, Negativo ou Zero

```c
#include <stdio.h>
#include <conio.h>
main() {
  int num;
  printf("Digite um numero: ");
  scanf("%d", &num);
  if (num > 0) {
    printf("O numero eh positivo");
  }
  else if (num < 0) {
    printf("O numero eh negativo");
  }
  else {
    printf("O numero eh zero");
  }
}
```

num > 0

Positivo ✓

num < 0

Negativo ✓

else

Zero ✓

Exemplo: uso completo de if/else if/else

Exemplo: Calculadora de Operações

```
float num1, num2, resultado;
int op;
scanf("%f %f", &num1, &num2);
printf ("menu: 1.Soma 2.Subtr. 3.Mult. 4.Div.
5.Pot.: ");
scanf("%d", &op);
if (op == 1)
  resultado = num1 + num2;
else if (op == 2)
  resultado = num1 - num2;
else if (op == 3)
  resultado = num1 * num2;
else if (op == 4)
  resultado = num1 / num2;
else if (op == 5)
  resultado = pow(num1, num2);
printf("Resultado: %f", resultado);
```

Cada opção corresponde a um bloco

Usa else if em cadeia para múltiplas condições

Nenhum else final — nenhuma mensagem de erro!

Usa pow() da biblioteca <math.h> para potência

03

OPERADORES LÓGICOS

&& || ! — combinando condições

Operadores Lógicos em C

&&

AND — E

TODOS os termos devem ser verdadeiros para o resultado ser verdadeiro

||

OR — OU

PELO MENOS UM termo deve ser verdadeiro para o resultado ser verdadeiro

!

NOT — NÃO

INVERTE ou NEGA o valor lógico de uma expressão ou elemento

Estes operadores permitem combinar múltiplos operadores relacionais em uma única condição

Tabela Verdade – AND (&&)

Para que o resultado seja VERDADEIRO, TODOS os termos precisam ser verdadeiros.

p	q	p && q
1 (V)	1 (V)	**1 (V)**
1 (V)	0 (F)	**0 (F)**
0 (F)	1 (V)	**0 (F)**
0 (F)	0 (F)	**0 (F)**

Regra do AND:

Basta UMA condição falsa para o resultado ser FALSO

Exemplo:

```
if (val > 0 && val % 2 == 0)
```

Positivo E par

Tabela Verdade – OR (||)

Para que o resultado seja VERDADEIRO, PELO MENOS UM termo precisa ser verdadeiro.

p	q	p || q
1 (V)	1 (V)	**1 (V)**
1 (V)	0 (F)	**1 (V)**
0 (F)	1 (V)	**1 (V)**
0 (F)	0 (F)	**0 (F)**

Regra do OR:

Basta UMA condição verdadeira para o resultado ser VERDADEIRO

Único FALSO:

Quando AMBAS as condições são falsas

$p = 0 \quad e \quad q = 0 \rightarrow 0$

Tabela Verdade – NOT (!)

O operador NOT inverte o valor lógico de uma expressão:

p	! p
1 (V)	**0 (F)**
0 (F)	**1 (V)**

Exemplos práticos:

```
!(val % 2 == 0)
```

val % 2 NÃO é igual a 0 → ímpar

```
!(x > 10)
```

x NÃO é maior que 10 → x <= 10

Equivalência:

```
val % 2 != 0 ≡ !(val % 2 == 0)   →  Ambas verificam se o número é ímpar!
```

Exemplo: Número Par e Positivo

```c
#include <stdio.h>
main() {
  int val;
  printf("Digite um número: ");
  scanf("%d", &val);
  if (val > 0 && val % 2 == 0) {
    printf("O número é par e positivo");
  }
}
```

Entendendo a condição:

`val > 0`

val é maior que zero?

`&&`

E (ambas devem ser verdadeiras)

`val % 2 == 0`

Resto da divisão por 2 é zero?

Só imprime se val for PAR e POSITIVO ao mesmo tempo!

Exemplos: Número Ímpar e Positivo

Ex. !=
```
if (val > 0 && val % 2 != 0)
printf("O número é ímpar e positivo");
```

Ex. !
```
if (val > 0 && !(val % 2 == 0))
printf("O número é ímpar e positivo");
```

Ex. !
```
if (val > 0 && !(val%2==0))
printf("O número é ímpar e positivo");
```

Equivalentes!: **Todas produzem o mesmo resultado**

Conclusão: `val % 2 != 0` ≡ `!(val % 2 == 0)` → Diferentes formas de verificar ímpar

04

ENCADEAMENTO DE CONDIÇÕES

if aninhado · seleção encadeada · desvio condicional composto

O que é Encadeamento?

É possível encadear desvios condicionais fazendo com que, após um teste verdadeiro, outro teste seja realizado.

Desvio condicional composto

Seleção encadeada

if aninhado (nested if)

```
if (condição1) {
  if (condição2) {
    // bloco 1
  }
  else {
    // bloco 2
  }
  // bloco 3
}
else {
  // bloco 4
}
```

Condição 1 avaliada primeiro

Se VERDADEIRA → condição 2 avaliada

Cada bloco executado apenas quando seu caminho for escolhido

Exemplo: if Aninhado — Comparar com 10

```c
#include <stdio.h>
main() {
  int num;
  printf("Digite um numero: ");
  scanf("%d", &num);
  if (num == 10) {
    printf("Voce acertou!");
    printf("O numero eh igual a 10.");
  }
  else {
    if (num > 10) {
      printf("O numero eh maior que 10.");
    }
    else {
      printf("O numero eh menor que 10.");
    }
  }
}
```

Árvore de decisão:

`num == 10?` → **Igual ✓**

`num > 10?` → **Maior ✓**

Menor ✓

3 resultados possíveis com apenas 2 condições!

Quando usar if, else, else if?

SEMPRE IRÁ DEPENDER DO PROBLEMA QUE ESTÁ SENDO RESOLVIDO!

`if`	Quando há uma única condição a verificar — executa bloco só se verdadeiro
`if + else`	Quando há dois caminhos: um para verdadeiro e outro para falso
`else if`	Quando há múltiplas condições mutuamente exclusivas em cadeia
`if aninhado`	Quando uma decisão depende do resultado de outra decisão anterior

Comparativo: Estruturas de Desvio

Somente if

```
if (a > 0) {
  // executa se a>0
}
if (a < 0) {
  // executa se a<0
}
// Ambos avaliados!
```

if + else if

```
if (a > 0) {
  // positivo
}
else if (a < 0) {
  // negativo
}
else {
  // zero
}
```

if aninhado

```
if (a >= 0) {
  if (a > 0) {
    // positivo
  }
  else {
    // zero
  }
}
```

Dica: else if é mais eficiente que múltiplos ifs quando as condições são exclusivas — evita testes desnecessários!

Exercício Guiado: Classificação de Notas

Problema: Receba uma nota (0–10) e classifique:

9 – 10	**5 – 6.9**
Excelente	**Regular**
7 – 8.9	**0 – 4.9**
Bom	**Reprovado**

```
float nota;
scanf("%f", &nota);
if (nota >= 9.0) {
  printf("Excelente");
}
else if (nota >= 7.0) {
  printf("Bom");
}
else if (nota >= 5.0) {
  printf("Regular");
}
else {
  printf("Reprovado");
}
```

Prioridade dos Operadores

Maior	( )	Parênteses — alteram a prioridade
↑	!	NOT — operador unário
↑	* / %	Multiplicação, divisão, módulo
↑	+ -	Adição e subtração
↑	< > <= >=	Relacionais de desigualdade
↑	== !=	Relacionais de igualdade
↑	&&	AND lógico
Menor	||	OR lógico

Erros Comuns — Cuidados!

```
✗ if (x = 10)
```

```
✓ if (x == 10)
```

Usar = (atribuição) em vez de == (comparação). O programa compilará mas com comportamento errado!

```
✗ if (x > 0)
```

```
✓ if (x > 0) {
```

Esquecer as chaves { } — apenas a primeira instrução faz parte do if!

```
✗ if (0 < x < 10)
```

```
✓ if (x > 0 && x < 10
```

Usar comparação encadeada — inválido em C! Use && para combinar condições.

Dicas para Depurar Condições

1. Teste com valores-limite: teste exatamente no valor da condição (ex: x = 10 quando condição é x > 10)
2. Use printf para imprimir valores antes do if e ver o que a variável contém
3. Desenhe a árvore de decisão no papel antes de programar
4. Sempre use { } mesmo para blocos de uma linha — evita bugs difíceis de encontrar
5. Verifique se está usando == (comparação) e não = (atribuição) dentro do if

Quiz Rápido!

1. O que imprime: `x=5; if(x>3) printf("A"); else printf("B");`

Resposta:

→ A

2. Resultado de: `0 && 1`

Resposta:

→ 0 (FALSO)

3. Resultado de: `0 || 1`

Resposta:

→ 1 (VERDADEIRO)

4. O que faz: `if(!0) printf("OK");`

Resposta:

→ Imprime OK

Aplicações Práticas no Dia a Dia

Login de Sistemas

```
if (senha == correta && usuario
== existe) → permite acesso
```

Jogos

```
if (vida <= 0) → game over
else → continua
```

Automação

```
if (temperatura > 30) → liga ar
condicionado
```

E-commerce

```
if (estoque > 0 &&
usuario_logado) → habilita compra
```

Resumão: Todos os Operadores

RELACIONAIS	LÓGICOS
`>` · `<` · `>=` · `<=` · `==` · `!=`	`&&` · `||` · `!`
Comparam valores → retornam 0 ou 1	*Combinam condições → retornam 0 ou 1*

```
// Combinando tudo numa única condição:
if ( x > 0 && x < 100 && x % 2 == 0 ) {
    printf("x é positivo, menor que 100 e par");
}

// Equivalente com NOT:
if ( x > 0 && x < 100 && !(x % 2 != 0) ) {
    printf("x é positivo, menor que 100 e par");
}
```

Exercícios Propostos

Ex. 1

Ano Bissexto

Leia um ano e informe se é bissexto.
Um ano é bissexto se divisível por 4 E (não divisível por 100 OU divisível por 400).
Dica: use % (módulo) e operadores && / ||

Ex. 2

IMC — Índice de Massa Corporal
Leia peso e altura, calcule o IMC (peso/altura²).
Classifique: < 18.5 = Baixo peso, 18.5–24.9 = Normal, 25–29.9 = Sobrepeso, >= 30 = Obeso.
Dica: use else if em cadeia

Ex. 3

Triângulo

Leia 3 lados. Primeiro verifique se formam um triângulo (cada lado < soma dos outros dois). Se sim, classifique: equilátero, isósceles ou escaleno.
Dica: use if aninhado — primeiro a validade, depois o tipo

Exercício Extra: Sistema de Menu

Implemente um menu de caixa eletrônico simplificado:

```
int opcao;
float saldo = 1500.0, valor;
printf("1. Consultar saldo\n");
printf("2. Sacar\n");
printf("3. Depositar\n");
scanf("%d", &opcao);
if (opcao == 1) {
    printf("Saldo: R$ %.2f\n", saldo);
}
else if (opcao == 2) {
    scanf("%f", &valor);
    if (valor <= saldo) {
        saldo -= valor;
        printf("Saque realizado! Saldo: %.2f", saldo);
    }
    else {
        printf("Saldo insuficiente!");
    }
}
else if (opcao == 3) { ... }
```

- if aninhado dentro do else if
- Valida saldo antes de sacar
- Condição composta com &&

Desafio: complete com depositar e uma opção 'else' para opção inválida!

Revisão Rápida

`if sem else`

Executa bloco só se condição VERDADEIRA

`&&` (AND)

Verdadeiro SOMENTE se TODOS forem V

`||` (OR)

Verdadeiro se PELO MENOS UM for V

`!` (NOT)

INVERTE o valor lógico

`==` vs `=`

== compara · = atribui (nunca confundir!)

`if aninhado`

Decisão DENTRO de outra decisão

O que aprendemos hoje

- Estrutura de desvio: if, else e else if para controle de fluxo
- Operadores relacionais: >, <, >=, <=, ==, != para comparações
- Operadores lógicos: && (AND), || (OR), ! (NOT) para combinar condições
- Encadeamento de condições: if aninhado para decisões complexas
- Em C, 0 = FALSO e qualquer outro número = VERDADEIRO

Boa prática!